

SmartBiz Partners — Product & Functional Requirements Document

Version: 1.0

Date: May 2026

Product: SmartBiz Partners

Live URL: <https://smartbiz.team>

TABLE OF CONTENTS

1. Product Requirements Document (PRD)

- Product Overview
- Problem Statement
- Target Users
- Goals & Success Metrics
- User Stories
- Non-Functional Requirements

1. Functional Requirements Document (FRD)

- System Architecture
 - Authentication & User Management
 - Dashboard
 - Invoices
 - Quotes
 - Expenses
 - CRM (Customers)
 - Inventory
 - Online Store
 - Orders
 - Reports & Analytics
 - AI Assistant
 - Admin Panel
 - Data Models
 - Integrations
 - Security Requirements
-

PART 1 — PRODUCT REQUIREMENTS DOCUMENT (PRD)

1. Product Overview

SmartBiz Partners is an all-in-one business management web application built specifically for Nigerian small and medium-sized enterprises (SMEs). It combines invoicing, expense tracking, CRM, inventory management, and an online storefront into a single, clean, mobile-friendly platform — removing the need for multiple disconnected tools.

Live at: <https://smartbiz.team>

Tech Stack: React 18, TypeScript, Vite, Supabase (PostgreSQL + Auth + Storage), Paystack, Tailwind CSS, shadcn/ui, Framer Motion

2. Problem Statement

Nigerian SMEs operate in a fragmented tool landscape:

- They send invoices via WhatsApp screenshots or Excel
- They track expenses in paper notebooks or personal spreadsheets
- They manage customer contacts in phone books or scattered notes
- They have no visibility into stock levels until items run out
- They have no affordable way to sell online without building a separate website

The result is lost revenue, disorganised operations, and no real data to make business decisions.

SmartBiz Partners solves this by providing a single, affordable platform designed for how Nigerian businesses actually operate.

3. Target Users

Primary Users — Small Business Owners

- Fashion designers, tailors, boutique owners
- Food vendors, caterers, bakeries
- Electronics & phone accessory retailers
- Beauty, hair, and cosmetics sellers
- Freelancers and service providers (photographers, web designers, consultants)
- Traders at markets and online

Secondary Users — Business Managers / Staff

- Employees who manage stock, process orders, or issue invoices on behalf of the business owner

Geographic Focus

- Nigeria (primary) — Naira (NGN) currency, Paystack payments, local context
-

4. Goals & Success Metrics

Business Goals

| Goal | Metric |

|-----|-----|

| Acquire SME users | Monthly active users (MAU) |

| Monetise through subscriptions | Paid subscriber count, MRR |

| Prove value before paywall | 14-day free trial → paid conversion rate |

| Build store payment GMV | Total store order volume (NGN) |

Product Goals

| Goal | Metric |

|-----|-----|

| Reduce time to first invoice | < 3 minutes from sign-up to first invoice sent |

| Drive store adoption | % of users who activate online store |

| Retain users long-term | Monthly retention rate |

| Keep app fast on mobile | Page load < 2s on 4G connection |

5. User Stories

Authentication

- As a new user, I can sign up with email + password and get a 14-day free trial
- As a returning user, I can sign in with email + password
- As a user, I can sign in with Google for one-click access
- As a Google user, I am prompted to enter my business name on first login
- As a user, I can reset my password via email
- As a signed-in user visiting the homepage, I am automatically redirected to my dashboard

Dashboard

- As a user, I can see my total revenue, pending invoices, expenses, and net profit at a glance
- As a user, I can see a to-do list of urgent items (unpaid invoices, low stock)
- As a user, I can upload my business logo
- As a user, I can update my business name, email, and phone in settings
- As a user, I can copy and share my store link directly from the dashboard

Invoices

- As a user, I can create a professional invoice with line items, tax, and discount
- As a user, I can auto-fill customer details from my CRM when creating an invoice
- As a user, I can mark invoices as draft, sent, paid, or overdue

- As a user, I can download/print a PDF of any invoice
- As a user, invoices are automatically numbered sequentially (e.g. INV-001)

Quotes

- As a user, I can create a quote (proposal) for a client
- As a user, I can convert a quote to an invoice in one click
- As a user, quotes are numbered with a QT- prefix (e.g. QT-001) and kept separate from invoices

Expenses

- As a user, I can log business expenses with description, amount, category, and date
- As a user, I can categorise expenses (Rent, Utilities, Transport, Salary, Marketing, etc.)
- As a user, I can see total expenses and this month's spend at the top of the page
- As a user, I can delete expense entries with a confirmation dialog

CRM (Customers)

- As a user, I can add customers with name, email, phone, company, and notes
- As a user, when creating an invoice or quote, I can pick a customer to auto-fill their details
- As a user, I can see total customers and how many were added this month
- As a user, I can search customers by name, email, or company
- As a user, I can delete a customer with a confirmation dialog

Inventory

- As a user, I can add products with name, SKU, category, quantity, unit price, and low-stock threshold
- As a user, I can see which products are running low on stock (highlighted in red)
- As a user, I can see the total stock value across all products
- As a user, I can publish any inventory item directly to my online store
- As a user, I can delete products with a confirmation dialog

Online Store

- As a user, I can create a public-facing store with a custom URL slug (e.g. /store/my-store)
- As a user, I can customise my store's appearance: theme color, logo, and banner image
- As a user, I can add products with images, prices, compare-at prices, categories, and stock quantities
- As a user, I can toggle individual products as visible or hidden
- As a user, I can publish or unpublish my store at any time
- As a user, I can copy my store link to share with customers

Store (Customer View — Public Storefront)

- As a customer, I can browse products in a published store without logging in
- As a customer, I can add items to cart and adjust quantities
- As a customer, I can enter my name, email, phone, and delivery address to check out

- As a customer, I am redirected to Paystack to complete payment
- As a customer, I am shown a success page after successful payment

Orders

- As a user, I can see all orders placed in my store
- As a user, I can click an order to see the customer's details and the items they ordered
- As a user, I can update an order's status (pending → confirmed → shipped → delivered → cancelled)
- As a user, I can see the payment status of each order (unpaid / paid)

Reports

- As a user, I can see charts of my revenue trend over the last 6 months
- As a user, I can see a breakdown of expenses by category (pie chart)
- As a user, I can see customer growth over the last 6 months
- As a user, I can see summary cards for paid revenue, pending invoices, total expenses, and total customers

AI Assistant

- As a user, I can chat with an AI business advisor for help with financial decisions, pricing, and business questions
- As a user, the AI has context about my business (name, industry)

Admin Panel (Admin users only)

- As a platform admin, I can view all registered users and their subscription statuses
- As a platform admin, I can see platform-wide usage statistics

6. Non-Functional Requirements

Performance

- Initial page load: < 2 seconds on 4G mobile
- Dashboard data load: < 3 seconds
- All SmartBooks pages use lazy loading (code splitting) to minimise bundle size

Availability

- Target uptime: 99.9% (relying on Supabase and Lovable infrastructure)

Mobile Responsiveness

- The entire app — dashboard, forms, dialogs, storefront — must be fully usable on mobile screens (minimum 375px width)

Security

- All user data isolated by `user_id` with Row Level Security (RLS) on every Supabase table
- No user can access another user's data via the API
- Auth handled by Supabase Auth (JWTs, secure sessions)
- Paystack payments handled server-side via Edge Functions (secret keys never exposed client-side)

Accessibility

- Keyboard navigable forms and dialogs
- Sufficient colour contrast on all text

Localisation

- Currency: Nigerian Naira (NGN) formatted with `Intl.NumberFormat("en-NG")`
- Date formats: local Nigerian date format
- Phone placeholder: +234 prefix

PART 2 — FUNCTIONAL REQUIREMENTS DOCUMENT (FRD)

1. System Architecture



▼

Paystack API
(payment initiation + webhook callbacks)

2. Authentication & User Management

Sign-Up (Email/Password)

1. User enters: business name (required), phone (optional), email, password (min 6 chars)
2. `supabase.auth.signUp()` called with `user_metadata: { business_name, phone }`
3. Supabase `handle_new_user` trigger fires → inserts row into `profiles` table
4. 14-day trial period begins (`trial_ends_at = now() + 14 days`)
5. Auth context detects user → navigates to `/smartbooks`

Sign-In (Email/Password)

1. User enters email + password
2. `supabase.auth.signInWithPassword()` called
3. On success, `onAuthStateChange` fires → updates auth context
4. `useEffect` in Auth page detects `user` → navigates to `/smartbooks`
5. `ProtectedRoute` verifies user before rendering SmartBooks

Sign-In (Google OAuth)

1. User clicks "Continue with Google"
2. `lovable.auth.signInWithOAuth("google", { redirect_uri: "/smartbooks" })` called
3. Google OAuth flow completes → redirects to `/smartbooks`
4. If profile has no `business_name` (empty string from trigger default), a non-dismissable dialog prompts the user to enter their business name before accessing the dashboard

Password Reset

1. User enters email on auth page → clicks "Forgot your password?"
2. `supabase.auth.resetPasswordForEmail()` sends reset link to email
3. Link redirects to `/reset-password` page
4. User sets new password

Session Persistence

- Sessions stored in `localStorage` via Supabase client config
- Auto-refresh tokens enabled
- Safety timeout: loading state never hangs beyond 3 seconds

Route Protection

- `/smartbooks/*` wrapped in `ProtectedRoute`
 - If `loading = true` → show spinner
 - If `user = null` → redirect to `/smartbooks/auth`
 - If `user` set → render page
-

3. Dashboard

Data Fetched

Parallel queries on mount (filtered by `user_id`):

- `invoices` — count, revenue (paid), pending (unpaid/draft/overdue) — **excludes QT- prefixed quotes**
- `expenses` — total spend
- `customers` — total count, new this month
- `inventory` — low stock count (items at or below threshold)
- `store_orders` — total orders, paid order count
- `store_settings` — store slug (for share link)

Sections Rendered

1. **Greeting banner** — business name, store share link (if store is live)
2. **Revenue card** — total revenue, pending, expenses, net profit (with show/hide toggle)
3. **To-Do list** — dynamic alerts: unpaid invoices, low stock, missing logo, store not set up
4. **Business overview** — 4 stat cards: orders, products sold, new customers, total clients
5. **Financial summary** — total sales, settled, owed, expenses
6. **Quick actions** — 8 navigation shortcuts to all SmartBooks sections
7. **Settings dialog** — business name, email, phone, logo upload

Logo Upload

- Uploaded to Supabase Storage bucket: `logos`
 - Path: `{user_id}/logo-{timestamp}.{ext}`
 - Max size: 5MB
 - Formats: image/*
 - URL saved to `profiles.logo_url`
-

4. Invoices

Invoice Creation

- Fields: customer (CRM picker), invoice number (auto-generated), issue date, due date, line items (description, quantity, unit price), discount (%), tax (%), notes, status
- Auto-fill: selecting a customer from CRM auto-fills name, email, phone, address
- Auto-numbering: sequential INV-001, INV-002, etc.
- Status options: draft, sent, paid, overdue

Invoice List

- Sorted by creation date (newest first)
- Filterable by status (all / paid / unpaid / overdue)
- Searchable by customer name or invoice number
- Summary stats: total revenue, pending, paid count

PDF/Print

- Invoice rendered as a printable HTML layout
- User can print via browser or save as PDF

Quote Distinction

- Quotes share the `invoices` table but use a `QT-` prefix on `invoice_number`
- All invoice queries filter out `QT-%` records to avoid count/revenue inflation

5. Quotes

Quote Creation

- Same fields as invoices
- Auto-numbered with QT- prefix (QT-001, QT-002, etc.)
- Status: draft, sent, accepted, rejected

Convert to Invoice

- One-click conversion: creates a new invoice record with all quote data
- Preserves customer details, line items, amounts
- Quote status updated to "accepted"

6. Expenses

Expense Entry

- Fields: description (required), amount (required), category (dropdown), date
- Categories: Rent, Utilities, Transport, Supplies, Marketing, Salary, Equipment, Food, Internet, Other

Summary Stats

- Total expenses (all time)
- This month's expenses (filtered by current month AND year)
- Total entry count

List View

- Sorted by date (newest first)
 - Searchable by description or category
 - Delete with confirmation dialog
-

7. CRM (Customers)

Customer Record

- Fields: name (required), email, phone, company, notes

Usage

- Customer list is accessible from Invoice and Quote creation forms
- Selecting a customer auto-fills: name, email, phone, address into the invoice/quote

Summary Stats

- Total customers
- Added this month (filtered by current month AND year)

Search

- Real-time filter by name, email, or company
-

8. Inventory

Product Record

- Fields: name (required), SKU, category, quantity, unit price, low-stock threshold (default 5)

Low Stock Alerts

- Products at or below their threshold shown with a warning icon (AlertTriangle)
- Dashboard to-do list surfaces the count of low-stock items

Total Value

- Calculated as: `SUM(quantity × unit_price)` across all products

List to Store

- Any inventory item can be published to the online store via a "List in Store" dialog
 - Sets store product price and description
 - Creates a new `store_products` record (does not modify inventory record)
-

9. Online Store

Store Settings

- **Store Info:** store name, URL slug (auto-generated, unique), description, published toggle
- **Appearance:**
 - Theme color: 9 preset swatches (Zinc, Blue, Indigo, Emerald, Orange, Red, Purple, Pink, Amber)
 - Store logo: uploadable image, shown in storefront header and status banner
 - Banner image: uploadable hero image shown below storefront header (recommended 1200×400px)
- **URL:** `{origin}/store/{slug}` — publicly accessible without login

Store Products

- Fields: name, description, price, compare-at price (for sale display), category, stock quantity, image, active/hidden toggle
- Images uploaded to Supabase Storage bucket: `store-images`
- Products can be edited, hidden/shown, or deleted from the dashboard

Store Appearance Applied

- Theme color: applied to "Add to Cart" buttons, product prices, cart badge, checkout total, Pay Now button
- Logo: shown in storefront header (circular or rounded square)
- Banner: full-width hero image below the header (height: 160px mobile, 224px desktop)

Storefront (Public — No Login Required)

- Accessible to any customer via the store URL
- Shows: store header (logo, name, description), banner, product grid
- Products filtered to: `is_active = true`
- Store settings filtered to: `is_published = true`

Checkout Flow

1. Customer adds products to cart
2. Cart dialog shows items, quantities, total
3. Customer proceeds to checkout

4. Fills in: name (required), email (required), phone (required), delivery address (optional)
5. Order record created in `store_orders`
6. Order items inserted into `store_order_items`
7. `store-checkout` Edge Function called → initialises Paystack transaction
8. Customer redirected to Paystack payment page
9. On success: redirected to `/store/order-success?order_id=...&slug=...`
10. Paystack webhook verifies payment → updates `store_orders.payment_status = "paid"`

Order Success Page

- Shows: success icon, order reference (first 8 chars of order ID), "Back to Store" button (if slug in URL)

10. Orders

Order List

- Shows: order number, customer name, date, payment status badge, order status badge, total amount
- Searchable by order number or customer name
- Sorted by creation date (newest first)

Order Detail Dialog

- Customer info: name, email, phone, delivery address
- Order items: product name, unit price × quantity, line total
- Order total
- Payment reference (Paystack reference)
- Status update buttons: pending, confirmed, shipped, delivered, cancelled

Status Update

- Updates `store_orders.status`
- Error shown to user if update fails (does not show success toast on failure)

11. Reports & Analytics

Summary Cards

| Card | Value |

|-----|-----|

| Paid Revenue | Sum of all paid invoice totals |

| Pending Value | Sum of all non-paid invoice totals |

| Expenses | Sum of all expense amounts |
| Customers | Total CRM contact count |

Revenue Trend Chart

- Bar chart: last 6 months
- Y-axis: NGN revenue from paid invoices
- X-axis: month labels
- Data source: `invoices` where `status = "paid"`, grouped by `issue_date` month/year

Expense Breakdown Chart

- Pie/donut chart: top 5 expense categories by spend
- Legend with category name and NGN amount
- Remaining categories grouped into "Other"

Customer Growth Chart

- Line chart: last 6 months
- Y-axis: new customers added per month
- X-axis: month labels

Data Filtering

- Invoice data excludes QT- prefixed records (quotes)
- All data scoped to `user_id`

12. AI Assistant

- Accessible as a floating chat widget on the landing page
- Powered by Supabase Edge Function: `chat`
- Uses Anthropic / OpenAI API (configured via Supabase secret)
- Business context injected into prompt: business name
- Handles: pricing advice, cash flow questions, business strategy, Nigerian market context

13. Admin Panel

Access Control

- Only users with `user_roles.role = "admin"` can access `/smartbooks/admin`
- `isAdmin` flag exposed via `useAuth()` hook
- Sidebar "Admin" section hidden for non-admin users

Features

- View all registered users
- See subscription status per user
- Platform-wide usage overview

14. Data Models

profiles

Column	Type	Notes
id	uuid	FK to auth.users
email	text	nullable
business_name	text	default ''
phone	text	nullable
logo_url	text	nullable
subscription_status	text	nullable
trial_ends_at	timestampz	nullable
updated_at	timestampz	

invoices

Column	Type	Notes
id	uuid	PK
user_id	uuid	FK to auth.users
invoice_number	text	INV-xxx or QT-xxx
customer_name	text	
customer_email	text	nullable
customer_phone	text	nullable
customer_address	text	nullable
issue_date	date	
due_date	date	nullable
items	jsonb	line items array
subtotal	numeric	
discount	numeric	
tax	numeric	
total	numeric	
status	text	draft/sent/paid/overdue/accepted/rejected
notes	text	nullable
created_at	timestampz	

customers

Column	Type	Notes
id	uuid	PK
user_id	uuid	FK
name	text	required
email	text	nullable
phone	text	nullable
company	text	nullable
notes	text	nullable
created_at	timestampz	

expenses

Column	Type	Notes
id	uuid	PK
user_id	uuid	FK
description	text	required
amount	numeric	
category	text	
date	date	
created_at	timestampz	

inventory

Column	Type	Notes
id	uuid	PK
user_id	uuid	FK
name	text	required
sku	text	nullable
quantity	integer	default 0
unit_price	numeric	default 0
category	text	nullable
low_stock_threshold	integer	default 5
created_at	timestampz	

store_settings

Column	Type	Notes
id	uuid	PK
user_id	uuid	FK (unique)
store_name	text	
store_slug	text	unique
description	text	
logo_url	text	nullable
banner_url	text	nullable
theme_color	text	default '#18181b'

currency	text	default 'NGN'
is_published	boolean	default false
created_at	timestampz	
updated_at	timestampz	

store_products

Column	Type	Notes
id	uuid	PK
user_id	uuid	FK
name	text	
description	text	
price	numeric	
compare_at_price	numeric	nullable
image_url	text	nullable
category	text	nullable
stock_quantity	integer	default 0
is_active	boolean	default true
created_at	timestampz	
updated_at	timestampz	

store_orders

Column	Type	Notes
id	uuid	PK
store_user_id	uuid	FK to store owner
order_number	text	ORD-{timestamp36}
customer_name	text	
customer_email	text	
customer_phone	text	nullable
customer_address	text	nullable
subtotal	numeric	
total	numeric	
status	text	pending/confirmed/shipped/delivered/cancelled
payment_status	text	unpaid/paid/refunded
payment_reference	text	nullable (Paystack ref)
notes	text	nullable
created_at	timestampz	
updated_at	timestampz	

store_order_items

Column	Type	Notes
id	uuid	PK
order_id	uuid	FK → store_orders (cascade delete)
product_id	uuid	FK → store_products

product_name text snapshot at order time
quantity integer
unit_price numeric
amount numeric quantity × unit_price
created_at timestamptz

15. Integrations

Supabase

- **Auth:** JWT-based sessions, Google OAuth, email/password, magic links
- **Database:** PostgreSQL with Row Level Security on all tables
- **Storage:** Two public buckets: `logos` (business logos), `store-images` (product/banner images)
- **Edge Functions (Deno runtime):**
 - `store-checkout` — initiates Paystack payment for store orders (JWT verification disabled for anonymous customers)
 - `paystack-checkout` — subscription billing
 - `paystack-webhook` — verifies Paystack events, updates payment status
 - `chat` — AI assistant proxy
 - `auth-email-hook` — custom auth email templates
 - `process-email-queue` — email delivery

Paystack

- Payment provider for Nigerian NGN transactions
- Store checkout: amount passed in kobo (NGN × 100)
- Callback URL: `{origin}/store/order-success?order_id={id}&slug={slug}`
- Secret key stored as Supabase secret: `paystack_API_KEY`

16. Security Requirements

Row Level Security (RLS)

All tables enforce user isolation:

Table authenticated can anon can
----- ----- -----
profiles SELECT/UPDATE own row —
invoices ALL on own rows —
customers ALL on own rows —
expenses ALL on own rows —
inventory ALL on own rows —
store_settings ALL on own rows SELECT published stores

store_products	ALL on own rows	SELECT active products
store_orders	SELECT/UPDATE own store orders	INSERT (place orders)
store_order_items	SELECT own order items	INSERT

Secret Management

- Paystack API key: Supabase Edge Function secret (never in client code)
- Supabase anon key: safe to expose (RLS enforces access control)
- Service role key: never used client-side

Auth Security

- Passwords: handled by Supabase Auth (bcrypt hashed)
- Sessions: stored in localStorage, auto-refreshed
- OAuth state: managed by Supabase + Lovable SDK
- Password reset: secure email link with expiry

Input Validation

- All user inputs trimmed before saving
- Numeric fields validated before DB insert
- File uploads: type and size checked client-side before upload (max 5MB, image/* only)
- Store slugs: sanitised to lowercase alphanumeric + hyphens only

17. Deployment

- **Hosting:** Lovable (<https://smartbiz.team>)
- **CI/CD:** Push to `main` branch on GitHub → Lovable auto-redeploys
- **Environment variables:** stored in `.env` (committed), read as `import.meta.env.VITE_*`
- **Supabase project:** `kjlpppxgydzdsmdowv`

Document prepared: May 2026 | SmartBiz Partners